

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-a4f95973c165df83a.jsonl`
- SHA-256: `42be04cee5dbe8721827e520dc48e7a6c5a846511ed95bfdbd6bfd47e504dcf2`
- Source modified: `2026-06-18T01:42:12+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:40:32.138Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.
```

CANDIDATE:

```
{
  "title": "Migration ladder not crash-safe: a mid-block interruption in the v3/v4 ALTER steps permanently bricks the DB",
  "file": "backend/infrastructure/database.py",
  "line": "156-157 (v3) and 168-169 (v4); failure window is between each ALTER and its own `PRAGMA user_version = N` at lines 159 and 178",
  "trigger": "Open a pre-v3 (or pre-v4) DB so the v3/v4 migration block runs, and have the process interrupted AFTER one of the non-idempotent `ALTER TABLE ... ADD COLUMN` statements commits but BEFORE that block's `PRAGMA user_version = N` is reached. Realistic causes: SIGKILL/power loss during the one-time upgrade, `disk I/O error`, or an `OperationalError` (e.g. busy_timeout exhausted) raised by a later statement in the same block. Concretely reproduced: start at user_version=3 with `videos` lacking `user_id`; run the v4 block; inject a failure right after `ALTER TABLE videos ADD COLUMN user_id`. Because Python's sqlite3 auto-commits DDL (across the supported 3.10-3.13 range), the column persists but user_version stays 3. On the NEXT open, `_init_db` sees version<4, re-runs `ALTER TABLE videos ADD COLUMN user_id`, and raises `sqlite3.OperationalError: duplicate column name: user_id`. `_init_db` is called from `__init__`, so every subsequent Database(db_path) construction throws and the app can never start against that DB.",
  "why_wrong": "A `PRAGMA user_version` migration ladder is expected to be crash-safe / re-runnable: a block must be re-applicable when interrupted before its version bump. The v1, v2, and v5 blocks honor this (all DDL is `CREATE ... IF NOT EXISTS`), but the v3 and v4 blocks use bare `ALTER TABLE ... ADD COLUMN` (no IF-NOT-EXISTS equivalent, no existence guard). The `with conn:` scope in `_connect` does NOT protect them, because in CPython's sqlite3 each ALTER/CREATE DDL statement commits independently of the surrounding block, so a half-applied block leaves the column present while user_version is unbumped. There is no rollback to a clean v(N-1) state and no recovery path; the DB is permanently unopenable.",
  "severity": "medium",
  "proposed_fix": "Make the v3/v4 ALTERs idempotent so a re-run after an interrupted upgrade is safe. Simplest: before each ALTER, check the column is absent, e.g. helper `def _has_col(cur, table, col): return col in {r[1] for r in cur.execute(f'PRAGMA table_info({table})')}` and guard `if not _has_col(cursor, 'jobs', 'policy'): cursor.execute('ALTER TABLE jobs ADD COLUMN policy TEXT')` (same for next_poll_at, videos.user_id, candidate_segments.user_id). Alternatively, scope each migration block in its own explicit BEGIN/COMMIT and only bump user_version inside
```

```
the same committed unit, or catch the 'duplicate column name' OperationalError per-ALTER and
continue. The IF-NOT-EXISTS index/table statements already in those blocks are fine.",
    "confidence": 0.7,
    "klass": "SQLite migration ladder & transactions"
}
```

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/backend/infrastructure/database.py and related code/tests READ-ONLY before judging.

LENS ? REFUTE: try hard to show it is intended / already guarded / tested. Default real=false unless you can prove it is a genuine bug.

2. assistant (2026-06-18T01:40:34.700Z)

I'll analyze this candidate bug about the SQLite migration ladder. Let me read the relevant code first.

3. user (2026-06-18T01:40:35.583Z)

```
1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
```

```

46         with self._connect() as conn:
47             cursor = conn.cursor()
48
49             # Videos Table
50             cursor.execute("""
51                 CREATE TABLE IF NOT EXISTS videos (
52                     id TEXT PRIMARY KEY,
53                     filepath TEXT NOT NULL,
54                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
55                     status TEXT NOT NULL,
56                     validation_results TEXT
57                 )
58             """)
59
60             # Play Segments Table (Extensible metadata JSON)
61             cursor.execute("""
62                 CREATE TABLE IF NOT EXISTS play_segments (
63                     id INTEGER PRIMARY KEY AUTOINCREMENT,
64                     video_id TEXT NOT NULL,
65                     start_time REAL NOT NULL,
66                     end_time REAL NOT NULL,
67                     duration REAL NOT NULL,
68                     server TEXT,
69                     receiver TEXT,
70                     metadata TEXT,
71                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72                 )
73             """)
74
75             # Events Table
76             cursor.execute("""
77                 CREATE TABLE IF NOT EXISTS events (
78                     id INTEGER PRIMARY KEY AUTOINCREMENT,
79                     segment_id INTEGER NOT NULL,
80                     timestamp REAL NOT NULL,
81                     type TEXT NOT NULL,
82                     player TEXT,
83                     details TEXT,
84                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
85             CASCADE
86                 )
87             """)
88
89             # Versioned migrations live here. PRAGMA user_version is the schema
90             # contract ? bump it for every change and add an ordered `if version < N`
91             # block below. Tests assert the expected version, so accidental drift fails
92             CI.
93             version = cursor.execute("PRAGMA user_version").fetchone()[0]
94
95             if version < 1:
96                 # v1: raw candidate detections (pre-confirmation), detector-agnostic.
97                 # Common fields are columns; detector-specific metrics go in `stats`
98                 JSON,
99                 # so a new segmenter reuses this table by setting its own
100                 `detector`/`stats`.
101                 cursor.execute("""
102                     CREATE TABLE IF NOT EXISTS candidate_segments (
103                         id INTEGER PRIMARY KEY AUTOINCREMENT,
104                         video_id TEXT NOT NULL,
105                         detector TEXT NOT NULL,
106                         chunk_index INTEGER,
107                         start_time REAL NOT NULL,
108                         end_time REAL NOT NULL,
109                         duration REAL NOT NULL,
110                         confidence REAL,

```

```

107         stats TEXT,
108         detection_params TEXT,
109         confirmed_segment_id INTEGER,
110         human_label TEXT,
111         notes TEXT,
112         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115     )
116     """)
117     cursor.execute("""
118         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119         ON candidate_segments(video_id, detector)
120     """)
121     cursor.execute("PRAGMA user_version = 1")
122
123     if version < 2:
124         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
result["status"].
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146         """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149
150     if version < 3:
151         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155         # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156         cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157         cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159         cursor.execute("PRAGMA user_version = 3")
160
161     if version < 4:

```

```

162 # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163 # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164 # path can scope rows to an owner. **NULL = local install = byte-
identical to
165 # today** ? every existing row stays NULL and every existing query
(which never
166 # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167 # Additive ONLY: no served-behavior change rides on this slice.
168 cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169 cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170 # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171 # of index churn while making the eventual per-user lookups cheap.
172 cursor.execute(
173     "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174     "WHERE user_id IS NOT NULL")
175 cursor.execute(
176     "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177     "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178 cursor.execute("PRAGMA user_version = 4")
179
180 if version < 5:
181     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
182     # version): the resolved per-user `fusion_config` overlay + an INLINE
183     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
184     # and provenance that earned it. `is_active` pins the one row the
serving bridge
185     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186     # only persists/loads it ? NOTHING reads it on the served path yet.
187     #
188     #   user_id           ? owner of the model (NULL allowed = the local
install)
189     #   version           ? monotonic per-user model version
190     #   baseline_version  ? the shared baseline this was fit against
(upgrade switch)
191     #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192     #   fusion_config     ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193     #   classifier_json   ? JSON LogisticRegression.to_dict() (None =
config-only model)
194     #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it
was promoted)
195     #   n_videos_at_fit   ? held-out-user corpus size when fit (min_n
trust floor input)
196     #   is_active        ? the one resolved row for this user (partial-
unique below)
197     cursor.execute("""
198         CREATE TABLE IF NOT EXISTS user_models (
199             id INTEGER PRIMARY KEY AUTOINCREMENT,
200             user_id TEXT,
201             version INTEGER NOT NULL DEFAULT 1,
202             baseline_version TEXT,
203             feature_schema_hash TEXT,
204             fusion_config TEXT,
205             classifier_json TEXT,
206             promotion_evidence TEXT,
207             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208             is_active INTEGER NOT NULL DEFAULT 0,

```

```

209             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
210         )
211     """
212     # One model version per user (NULLs compare distinct in SQLite, so the
local
213     # install can still carry multiple versioned rows; the active-row guard
below
214     # is what enforces a single served model).
215     cursor.execute("""
216         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217             ON user_models(user_id, version)
218     """)
219     # At most ONE active model per user: a partial-unique index over the
active rows.
220     # (SQLite treats NULL user_id rows as distinct, so the local install's
single
221     # active row is still guarded ? there is one local user.)
222     cursor.execute("""
223         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
224             ON user_models(user_id) WHERE is_active = 1
225     """)
226     cursor.execute("PRAGMA user_version = 5")
227     # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
228
229     conn.commit()
230
231     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
232         """Register or update a video row WITHOUT touching its child segments.
233
234         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
235         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
236         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
237         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
238         row in place; clearing segments is now an explicit, deliberate operation
239         (clear_video_data / prune_segments).
240         """
241         try:
242             with self._connect() as conn:
243                 conn.cursor().execute(
244                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
245                     "ON CONFLICT(id) DO UPDATE SET "
246                     "filepath=excluded.filepath, status=excluded.status, "
247                     "validation_results=excluded.validation_results",
248                     (video_id, filepath, status, json.dumps(validation_results))
249                 )
250                 conn.commit()
251             return True
252         except Exception as e:
253             logger.error(f"Failed to add video: {e}")
254             return False
255
256     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
257         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
258
259         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
260         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
261         """
262         with self._connect() as conn:

```

```

263         cur = conn.cursor()
264         p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
265                         (video_id,)).fetchone()[0]
266         c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
267                         (video_id,)).fetchone()[0]
268         return int(p), int(c)
269
270     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
271                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
272                       cand_after: Optional[int] = None) -> None:
273         """Delete play/candidate segments by id range (events cascade from
play_segments).
274
275         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
276         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
277         and keep the OLD ones when the run failed/produced nothing (`*_after`).
278         """
279         with self._connect() as conn:
280             cur = conn.cursor()
281             if play_upto is not None:
282                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
283             if play_after is not None:
284                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
285             if cand_upto is not None:
286                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
287             if cand_after is not None:
288                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
289             conn.commit()
290
291     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
292         try:
293             with self._connect() as conn:
294                 cursor = conn.cursor()
295                 cursor.execute(
296                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
298                 )
299                 conn.commit()
300                 return cursor.lastrowid
301         except Exception as e:
302             logger.error(f"Failed to add play segment: {e}")
303             return None
304
305     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
306         try:
307             with self._connect() as conn:
308                 conn.cursor().execute(
309                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
310                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
311                 )

```

```

312         conn.commit()
313     return True
314 except Exception as e:
315     logger.error(f"Failed to add event: {e}")
316     return False
317
318     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
319                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
320                               stats: Optional[Dict[str, Any]] = None,
321                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
322         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
323         detector-specific dicts stored as JSON. Returns the new candidate id, or
324         None."""
325         try:
326             with self._connect() as conn:
327                 cursor = conn.cursor()
328                 cursor.execute(
329                     """INSERT INTO candidate_segments
330                     (video_id, detector, chunk_index, start_time, end_time, duration,
331                     confidence, stats, detection_params)
332                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
333                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
334                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
335                 conn.commit()
336                 return cursor.lastrowid
337         except Exception as e:
338             logger.error(f"Failed to add candidate segment: {e}")
339             return None
340
341     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
342         """Record that a candidate detection was confirmed as a given play_segment."""
343         try:
344             with self._connect() as conn:
345                 conn.cursor().execute(
346                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
347                     ?",
348                     (segment_id, candidate_id))
349                 conn.commit()
350             return True
351         except Exception as e:
352             logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
353             return False
354
355     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
only_unlabeled: bool = False) -> List[Dict[str, Any]]:
356         """Fetch candidate detections for the annotation / fine-tuning workflow.
357         `stats` and `detection_params` are deserialized from JSON."""
358         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
359         params: List[Any] = [video_id]
360
361         if detector:
362             query += " AND detector = ?"
363             params.append(detector)
364         if only_unlabeled:
365             query += " AND human_label IS NULL"
366
367         query += " ORDER BY start_time"

```



```

369
370     candidates = []
371     with self._connect() as conn:
372         rows = conn.cursor().execute(query, params).fetchall()
373         for row in rows:
374             d = dict(row)
375             d["stats"] = json.loads(d["stats"] or "{}")
376             d["detection_params"] = json.loads(d["detection_params"] or "{}")
377             candidates.append(d)
378     return candidates
379
380     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
381         with self._connect() as conn:
382             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
383 (video_id,)).fetchone()
384             if row:
385                 d = dict(row)
386                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
387                 return d
388             return None
389
390     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
391         """
392         Dynamically queries play segments based on filters:
393         - duration_min: float
394         - server: string
395         - receiver: string
396         - event_type: string (e.g. smash, foul)
397         """
398         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
399         params = [video_id]
400
401         if filters.get("duration_min"):
402             query += " AND r.duration >= ?"
403             params.append(filters["duration_min"])
404
405         if filters.get("server"):
406             query += " AND r.server = ?"
407             params.append(filters["server"])
408
409         if filters.get("receiver"):
410             query += " AND r.receiver = ?"
411             params.append(filters["receiver"])
412
413         if filters.get("event_type"):
414             # Check if there is an event of a specific type inside this segment
415             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
416             params.append(filters["event_type"])
417
418         segments_list = []
419         with self._connect() as conn:
420             cursor = conn.cursor()
421             rows = cursor.execute(query, params).fetchall()
422
423             for row in rows:
424                 r_dict = dict(row)
425                 r_id = r_dict["id"]
426                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
427
428                 # Fetch associated events
429                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
430                 r_dict["events"] = []

```

```

430         for er in event_rows:
431             e_dict = dict(er)
432             e_dict["details"] = json.loads(e_dict["details"] or "{}")
433             r_dict["events"].append(e_dict)
434
435         segments_list.append(r_dict)
436
437     return segments_list
438
439     def clear_video_data(self, video_id: str):
440         with self._connect() as conn:
441             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442             conn.commit()
443
444     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
445     # Writers follow the repo convention (swallow + logger.error, return bool); the
446     # worker logs loudly on a failed transition so a job can't silently wedge.
447
448     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                   player_pool: Optional[List[str]] = None,
450                   max_frames: Optional[int] = None,
451                   policy: Optional[Dict[str, Any]] = None) -> bool:
452         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
453         ExecutionPolicy
454         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
455         try:
456             with self._connect() as conn:
457                 conn.cursor().execute(
458                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
459 max_frames, status, policy) "
460                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
461                     (job_id, video_path, segmenter,
462                      json.dumps(player_pool) if player_pool is not None else None,
463                      max_frames, json.dumps(policy) if policy is not None else None)
464                 )
465                 conn.commit()
466             return True
467         except Exception as e:
468             logger.error(f"Failed to create job {job_id}: {e}")
469             return False
470
471     def mark_job_running(self, job_id: str) -> bool:
472         try:
473             with self._connect() as conn:
474                 conn.cursor().execute(
475                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
476                     "progress='starting' WHERE id = ?", (job_id,))
477                 conn.commit()
478             return True
479         except Exception as e:
480             logger.error(f"Failed to mark job {job_id} running: {e}")
481             return False
482
483     def set_job_progress(self, job_id: str, progress: str) -> bool:
484         try:
485             with self._connect() as conn:
486                 conn.cursor().execute(
487                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
488                 conn.commit()
489             return True
490         except Exception as e:
491             logger.error(f"Failed to set progress for job {job_id}: {e}")
492             return False
493
494     def set_job_video_id(self, job_id: str, video_id: str) -> bool:

```

```

493         try:
494             with self._connect() as conn:
495                 conn.cursor().execute(
496                     "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
497                 conn.commit()
498             return True
499         except Exception as e:
500             logger.error(f"Failed to set video_id for job {job_id}: {e}")
501             return False
502
503     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
504         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
505         validation) is inside `result` ? job status 'done' means 'the worker
506         finished'."""
507         try:
508             with self._connect() as conn:
509                 conn.cursor().execute(
510                     "UPDATE jobs SET status='done', progress='finished', result=?, "
511                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
512                     (json.dumps(result), job_id))
513                 conn.commit()
514             return True
515         except Exception as e:
516             logger.error(f"Failed to mark job {job_id} done: {e}")
517             return False
518
519     def mark_job_failed(self, job_id: str, error: str) -> bool:
520         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
521         try:
522             with self._connect() as conn:
523                 conn.cursor().execute(
524                     "UPDATE jobs SET status='failed', error=?, "
525                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
526                 conn.commit()
527             return True
528         except Exception as e:
529             logger.error(f"Failed to mark job {job_id} failed: {e}")
530             return False
531
532     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
533         """Fetch one job; `result`/`player_pool` JSON deserialized."""
534         with self._connect() as conn:
535             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
536             (job_id,)).fetchone()
537             if not row:
538                 return None
539             d = dict(row)
540             d["result"] = json.loads(d["result"]) if d["result"] else None
541             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
542             None
543             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
544             return d
545
546     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
547     -----
548     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
549         """Attach/replace a job's JSON ExecutionPolicy."""
550         try:
551             with self._connect() as conn:
552                 conn.cursor().execute(
553                     "UPDATE jobs SET policy=? WHERE id = ?",
554                     (json.dumps(policy) if policy is not None else None, job_id))
555                 conn.commit()
556             return True
557         except Exception as e:

```

```

554         logger.error(f"Failed to set policy for job {job_id}: {e}")
555         return False
556
557     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
558         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
559         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
560         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
561         try:
562             with self._connect() as conn:
563                 conn.cursor().execute(
564                     "UPDATE jobs SET status='waiting', "
565                     "next_poll_at=datetime('now', ?), "
566                     "progress=COALESCE(?, progress) WHERE id = ?",
567                     (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
568                 conn.commit()
569             return True
570         except Exception as e:
571             logger.error(f"Failed to set job {job_id} waiting: {e}")
572             return False
573
574     def seconds_until_next_due(self) -> Optional[float]:
575         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
576         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
577         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578         try:
579             with self._connect() as conn:
580                 row = conn.cursor().execute(
581                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
582                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
583                     "FROM jobs WHERE status='waiting').fetchone()
584                 if row is None or row["secs"] is None:
585                     return None
586                 return max(0.0, float(row["secs"]))
587         except Exception as e:
588             logger.error(f"Failed to compute next-due delay: {e}")
589             return None
590
591     def claim_due_job(self) -> Optional[Dict[str, Any]]:
592         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
593         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
594         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
595         first. Returns the claimed job dict, or None if nothing is runnable."""
596         try:
597             with self._connect() as conn:
598                 cur = conn.cursor()
599                 row = cur.execute(
600                     "SELECT id, status FROM jobs WHERE status='queued' "
601                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
602                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
603                 if not row:
604                     return None
605                 cur.execute(

```

```

606         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
607         "WHERE id=? AND status=?", (row["id"], row["status"]))
608         conn.commit()
609         if cur.rowcount != 1:
610             return None                    # lost the race to a concurrent
worker
611         return self.get_job(row["id"])
612     except Exception as e:
613         logger.error(f"Failed to claim a due job: {e}")
614         return None
615
616     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
617         """Crash recovery: any job still queued/running from a previous process is dead
618         (the executor is in-process). Mark failed so clients see a terminal state, not a
619         hang. Called once at startup. Returns the number of jobs swept."""
620         try:
621             with self._connect() as conn:
622                 cur = conn.cursor()
623                 cur.execute(
624                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
625                     "WHERE status IN ('queued', 'running')", (reason,))
626                 conn.commit()
627                 return cur.rowcount
628         except Exception as e:
629             logger.error(f"Failed to sweep stale jobs: {e}")
630             return 0
631
632     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
633     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
634     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
635     # serving.py) resolves them, and wiring that bridge into the production decision
seam
636     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
637     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
638     # JSON-serialised dicts; the rest are scalar columns.
639
640     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
641                           baseline_version: Optional[str] = None,
642                           feature_schema_hash: Optional[str] = None,
643                           fusion_config: Optional[Dict[str, Any]] = None,
644                           classifier_json: Optional[Dict[str, Any]] = None,
645                           promotion_evidence: Optional[Dict[str, Any]] = None,
646                           n_videos_at_fit: int = 0,
647                           is_active: bool = False) -> Optional[int]:
648         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
649
650         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
651         rowid). When `is_active` is True this row becomes the user's single active
model ?
652         any previously-active row for the SAME user is first cleared, so the
653         `idx_user_models_one_active` partial-unique invariant always holds (one active
model
654         per user). Returns the row id, or None on failure.
655         """
656         try:
657             with self._connect() as conn:
658                 cur = conn.cursor()
659                 if is_active:
660                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
661                     if user_id is None:
662                         cur.execute(
663                             "UPDATE user_models SET is_active = 0 "

```

```

664         "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
665         (version,))
666     else:
667         cur.execute(
668             "UPDATE user_models SET is_active = 0 "
669             "WHERE user_id = ? AND is_active = 1 AND version != ?",
670             (user_id, version))
671     cur.execute(
672         """INSERT INTO user_models
673         (user_id, version, baseline_version, feature_schema_hash,
674         fusion_config, classifier_json, promotion_evidence,
675         n_videos_at_fit, is_active)
676         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
677         ON CONFLICT(user_id, version) DO UPDATE SET
678         baseline_version=excluded.baseline_version,
679         feature_schema_hash=excluded.feature_schema_hash,
680         fusion_config=excluded.fusion_config,
681         classifier_json=excluded.classifier_json,
682         promotion_evidence=excluded.promotion_evidence,
683         n_videos_at_fit=excluded.n_videos_at_fit,
684         is_active=excluded.is_active""",
685         (user_id, version, baseline_version, feature_schema_hash,
686         json.dumps(fusion_config) if fusion_config is not None else None,
687         json.dumps(classifier_json) if classifier_json is not None else
688         None,
689         json.dumps(promotion_evidence) if promotion_evidence is not None
690         else None,
691         n_videos_at_fit, 1 if is_active else 0))
692     conn.commit()
693     rid = cur.execute(
694         "SELECT id FROM user_models WHERE "
695         + ("user_id IS NULL" if user_id is None else "user_id = ?")
696         + " AND version = ?",
697         ((version,) if user_id is None else (user_id, version))).fetchone()
698     return int(rid[0]) if rid else None
699 except Exception as e:
700     logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
701     {e}")
702     return None
703
704 def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
705     """Return the single active per-user model row for ``user_id`` (NULL = local
706     install),
707     with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
708     deserialised,
709     or None if the user has no active model. Does not touch the served pipeline."""
710     with self._connect() as conn:
711         if user_id is None:
712             row = conn.cursor().execute(
713                 "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
714                 1").fetchone()
715         else:
716             row = conn.cursor().execute(
717                 "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
718                 (user_id,)).fetchone()
719         if not row:
720             return None
721         d = dict(row)
722         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
723             d[col] = json.loads(d[col]) if d[col] else None
724         return d

```

Let me examine the key claim of this candidate: whether the migration ladder is crash-safe and whether tests cover this. Let me look at the tests for the database and how `\_init\_db` / `\_connect` behaves with respect to transactions.

5. user (2026-06-18T01:40:41.171Z)

Found 25 files

- .claude\worktrees\hunt-base\tests\test_job_worker_loop.py
- .claude\worktrees\hunt-base\tests\test_database.py
- .claude\worktrees\hunt-base\tests\test_async_jobs.py
- .claude\worktrees\hunt-base\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
- .claude\worktrees\hunt-base\docs\archives\decisions\DECISIONS.md
- .claude\worktrees\hunt-base\docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
- .claude\worktrees\hunt-base\docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
- .claude\worktrees\hunt-base\docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
- .claude\worktrees\hunt-base\docs\UI_PROPOSAL.md
- .claude\worktrees\hunt-base\docs\UI_ALPHA_EXECUTION_PLAN.md
- .claude\worktrees\hunt-base\docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
- .claude\worktrees\hunt-base\docs\PLATFORM_ARCHITECTURE.md
- .claude\worktrees\hunt-base\docs\PER_VIDEO_OUTPUT_LAYOUT.md
- .claude\worktrees\hunt-base\docs\PERSONALIZATION_PLAN.md
- .claude\worktrees\hunt-base\docs\NEXT_STEPS.md
- .claude\worktrees\hunt-base\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
- .claude\worktrees\hunt-base\docs\GOLDEN_REGRESSION_FIXTURES.md
- .claude\worktrees\hunt-base\docs\EXECUTION_POLICY.md
- .claude\worktrees\hunt-base\docs\CODE_MAP.md
- .claude\worktrees\hunt-base\docs\COMMERCIALIZATION.md
- .claude\worktrees\hunt-base\docs\BACKLOG.md
- .claude\worktrees\hunt-base\docs\ANALYZERS_AND_RECONCILERS.md
- .claude\worktrees\hunt-base\backend\infrastructure\database.py
- .claude\worktrees\hunt-base\backend\config\models.py
- .claude\worktrees\hunt-base\CLAUDE.md

6. user (2026-06-18T01:40:41.279Z)

- .claude\worktrees\bhi-execpolicy\tests\test_database.py
- tests\test_database.py
- .claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_database.py
- .claude\worktrees\local-noai\tests\test_database.py
- .claude\worktrees\docs-rally-quality\tests\test_database.py
- .claude\worktrees\tier0-eval\tests\test_database.py
- .claude\worktrees\tier1-windowing\tests\test_database.py
- .claude\worktrees\w2-netcross\tests\test_database.py
- .claude\worktrees\w2-safeguard\tests\test_database.py
- .claude\worktrees\hardchop\tests\test_database.py
- .claude\worktrees\docs-today\tests\test_database.py
- .claude\worktrees\docs-golden\tests\test_database.py
- .claude\worktrees\r2-impl\tests\test_database.py
- .claude\worktrees\r4-endrule\tests\test_database.py
- .claude\worktrees\boxhill17\tests\test_database.py
- .claude\worktrees\pr-stitch\tests\test_database.py
- .claude\worktrees\hunt-base\tests\test_database.py
- .claude\worktrees\gap-doc\tests\test_database.py

7. assistant (2026-06-18T01:40:43.528Z)

Let me look at the migration tests to understand what's tested, especially any partial/interrupted migration scenarios.

8. user (2026-06-18T01:40:45.358Z)

```
80: def test_schema_version_is_current(tmp_path):
84:     version = conn.execute("PRAGMA user_version").fetchone()[0]
```

```

86:     # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
87:     assert version == 5
90: def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
92:     # Simulate a pre-v1 database that only has the old tables and no version.
100:    # ...and running init again must be a no-op (idempotent).
106:    version = conn.execute("PRAGMA user_version").fetchone()[0]
111:    assert version == 5          # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
210: def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212:    columns/tables appear, user_id back-fills to NULL, and prior rows survive the ALTERs.
214:    db = Database(str(db_path)) # fresh DB is born at the current version; simulate a v3
state
215:    # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
222:    # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind version
224:    conn.execute("ALTER TABLE videos DROP COLUMN user_id")
225:    conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
226:    conn.execute("PRAGMA user_version = 3")
231:    # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
234:    assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
254:    "id", "user_id", "version", "baseline_version", "feature_schema_hash",
263:    "user-1", version=1, baseline_version="base-2026-06",
275:    assert m["version"] == 1
276:    assert m["baseline_version"] == "base-2026-06"
290:    db.upsert_user_model("user-2", version=1, is_active=False)
294: def test_upsert_is_idempotent_on_user_version(tmp_path):
296:    rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
298:    "user-3", version=1, n_videos_at_fit=7, is_active=True,
301:    # Same (user, version) ? same row updated in place, not a duplicate.
314: def test_activating_new_version_demotes_prior_active(tmp_path):
316:    db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
317:    db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)
320:    assert active["version"] == 2 # the newer model is the single active one
322:    # Exactly one active row for the user (the partial-unique invariant holds).
333:    db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
346:    db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},

```

9. assistant (2026-06-18T01:40:49.468Z)

The tests cover the happy-path migration (v3?v5 preserving rows) and idempotency of a **complete** re-run, but NOT a partial/interrupted migration. Let me read the migration test in detail and understand the transaction semantics precisely.

10. user (2026-06-18T01:40:51.172Z)

```

80     def test_schema_version_is_current(tmp_path):
81         db_path = tmp_path / "test.db"
82         Database(str(db_path))
83         conn = sqlite3.connect(str(db_path))
84         version = conn.execute("PRAGMA user_version").fetchone()[0]
85         conn.close()
86         # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
87         assert version == 5
88
89
90     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
91         db_path = tmp_path / "legacy.db"
92         # Simulate a pre-v1 database that only has the old tables and no version.
93         conn = sqlite3.connect(str(db_path))
94         conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
95         conn.commit()
96         conn.close()
97
98         # Opening with the new Database should add candidate_segments without error...
99         Database(str(db_path))
100        # ...and running init again must be a no-op (idempotent).

```



```

101 Database(str(db_path))
102
103 conn = sqlite3.connect(str(db_path))
104 tables = [r[0] for r in conn.execute(
105     "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
106 version = conn.execute("PRAGMA user_version").fetchone()[0]
107 conn.close()
108 assert "candidate_segments" in tables
109 assert "jobs" in tables # v2 (async job model, #16)
110 assert "user_models" in tables # v5 (per-user model store)
111 assert version == 5 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
112
113
114 def test_candidate_round_trip(tmp_path):
115     db = Database(str(tmp_path / "test.db"))
116     db.add_video("vid1", "vid1.mp4", "processed", [])
117
118     cid = db.add_candidate_segment(
119         "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
120         stats={"peak_velocity": 0.8, "track_id_count": 2},
121         detection_params={"frame_skip": 3},
122     )
123     assert cid is not None
124
125     cands = db.query_candidate_segments("vid1")
126     assert len(cands) == 1
127     c = cands[0]
128     assert c["detector"] == "yolo_hybrid"
129     assert c["start_time"] == 10.0
130     assert c["duration"] == 15.0
131     assert c["confidence"] == 0.42
132     assert c["stats"]["peak_velocity"] == 0.8
133     assert c["detection_params"]["frame_skip"] == 3
134     assert c["confirmed_segment_id"] is None
135
136     # detector filter
137     assert db.query_candidate_segments("vid1", detector="motion") == []
138     assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
139
140
141 def test_candidate_link_and_unlabeled_filter(tmp_path):
142     db = Database(str(tmp_path / "test.db"))
143     db.add_video("vid1", "vid1.mp4", "processed", [])
144
145     cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
146     sid = db.add_play_segment("vid1", 1.0, 4.0)
147
148     assert db.link_candidate_to_segment(cid, sid) is True
149     assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
150
151     # only_unlabeled returns it while human_label is NULL
152     assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
153     with db._get_connection() as conn:
154         conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
155             ("true_rally", cid))
156         conn.commit()
157     assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
158
159
160 def test_candidate_cascade_on_video_delete(tmp_path):
161     db = Database(str(tmp_path / "test.db"))
162     db.add_video("vid1", "vid1.mp4", "processed", [])
163     db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
164
165     db.clear_video_data("vid1")

```

```

166         assert db.query_candidate_segments("vid1") == []
167
168
169     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
170
171     def _table_cols(db_path, table):
172         conn = sqlite3.connect(str(db_path))
173         cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
174         conn.close()
175         return cols
176
177
178     def test_v4_user_id_columns_present_and_nullable(tmp_path):
179         db_path = tmp_path / "test.db"
180         Database(str(db_path))
181         assert "user_id" in _table_cols(db_path, "videos")
182         assert "user_id" in _table_cols(db_path, "candidate_segments")
183
184         # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
185         conn = sqlite3.connect(str(db_path))
186         for table in ("videos", "candidate_segments"):
187             by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
188             assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
189         conn.close()
190
191
192     def test_v4_existing_writers_leave_user_id_null(tmp_path):
193         """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
194         row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
195         db_path = tmp_path / "test.db"
196         db = Database(str(db_path))
197         db.add_video("vid1", "vid1.mp4", "processed", [])
198         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
199
200         conn = sqlite3.connect(str(db_path))
201         vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1']").fetchone()[0]
202         cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1']").fetchone()[0]
203         conn.close()
204         assert vu is None
205         assert cu is None
206         # Existing readers are unaffected (no user_id filter in the query path).
207         assert len(db.query_candidate_segments("vid1")) == 1
208
209
210     def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
211         """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
212         columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERS."""
213         db_path = tmp_path / "legacy_v3.db"
214         db = Database(str(db_path)) # fresh DB is born at the current version; simulate a
v3 state
215         # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
216         db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
217         db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
218         conn = sqlite3.connect(str(db_path))
219         conn.execute("DROP INDEX IF EXISTS idx_videos_user")

```

Now I understand the test coverage clearly. The tests verify happy-path migration and full re-run idempotency, but do NOT cover an interrupted migration. Let me now precisely verify the central technical claim: does Python's sqlite3 (default isolation_level) auto-commit DDL statements, leaving them persisted even if a later statement in the same `with conn:` block fails?

This is the crux. Let me verify the actual sqlite3 transactional behavior on this environment.

12. user (2026-06-18T01:41:01.604Z)

```
python 3.13.13 sqlite 3.50.4
block failed: simulated SIGKILL/IO error mid-block
after failed block -> cols: ['id', 'policy'] user_version: 3
```

13. assistant (2026-06-18T01:41:05.298Z)

The first part of the mechanism is confirmed: on Python 3.13 / SQLite 3.50, a DDL `ALTER TABLE ... ADD COLUMN` persists even when a later statement in the same `with conn:` block raises and triggers a rollback. The column stays, but `user\_version` is NOT bumped.

Now the critical second part: does the re-run on next open actually raise `duplicate column name` and break construction? Let me simulate the full re-open path with the actual logic.

14. user (2026-06-18T01:41:11.131Z)

```
first open interrupted: interrupt
second open RAISED OperationalError: duplicate column name: policy
```

15. assistant (2026-06-18T01:41:22.741Z)

Both halves of the mechanism are confirmed. The candidate's technical claims are accurate:

1. DDL `ALTER TABLE` auto-commits independently of the `with conn:` rollback scope.
2. A mid-block interruption persists the column but leaves `user\_version` unbumped.
3. The next open re-runs the same block and raises `OperationalError: duplicate column name`, which propagates out of `\_init\_db` ? `\_\_init\_\_`, bricking every subsequent `Database()` construction against that DB.

Now I must apply the REFUTE lens rigorously. The candidate is technically correct about the mechanism. The real question for "is this a genuine defect a maintainer would call real" hinges on **reachability of the trigger**. Let me assess this carefully.

The key consideration: the failure window requires a process interruption (SIGKILL/power loss/disk-IO-error/lock-timeout) that lands **precisely** between two adjacent `cursor.execute()` calls in the v3 or v4 block. Let me think about whether this is a realistically reachable, real-world defect or a theoretical-only one, and check whether the project's conventions or context already address it.

Let me check how the DB is opened in production and whether concurrent writers / locks are a realistic trigger (the candidate mentions busy_timeout exhaustion as one path that raises OperationalError from a *later* statement in the same block).

16. user (2026-06-18T01:41:25.038Z)

```
.claude\worktrees\hunt-base\backend\main.py:782: db_instance = Database(db_path)
.claude\worktrees\hunt-base\backend\infrastructure\database.py:12: self._init_db()
.claude\worktrees\hunt-base\backend\infrastructure\database.py:44: def _init_db(self):
```

17. user (2026-06-18T01:41:27.580Z)

```
770
771 # Initialize Global Config and DB
772 global global_config, db_instance
773 global_config = load_app_config() # returns typed AppConfig
```

```

774
775     # The CLI --output-dir overrides the configured output directory. It now lives
776     # on the typed model (OutputConfig.output_dir), so every consumer ? including
777     # /api/compile ? reads the same value instead of a write-only runtime hack.
778     global_config.output.output_dir = args.output_dir
779     os.makedirs(global_config.output.output_dir, exist_ok=True)
780
781     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782     db_instance = Database(db_path)
783
784     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785     # the executor is in-process. Mark them failed so pollers see a terminal state.
786     swept = db_instance.fail_stale_jobs()
787     if swept:
788         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790     # CLI processing mode (no web UI needed)
791     if args.video_path and not args.server:
792         print(f"[CLI] Processing video file: {args.video_path}")
793         if not os.path.exists(args.video_path):
794             print(f"[FAIL] Video file not found: {args.video_path}")
795             return
796
797         video_id = compute_video_id(args.video_path)
798         was_reingest = db_instance.get_video(video_id) is not None
799

```

18. assistant (2026-06-18T01:41:45.977Z)

Now I have the full picture. Let me apply the REFUTE lens with discipline and weigh the verdict.

****The mechanism is real and confirmed**** ? both halves reproduce. This is not a false claim about SQLite/Python behavior. The question is whether it rises to "a genuine defect a maintainer would call real," judged under the REFUTE lens (default real=false unless proven).

****Arguments the candidate is REAL:****

- The v1, v2, v5 blocks ARE crash-safe (all `CREATE ... IF NOT EXISTS`), establishing that crash-safe/re-runnable migration was the clear design intent. The v3/v4 bare `ALTER` blocks break that invariant ? a genuine inconsistency, not a non-goal.
- The failure is non-recoverable and catastrophic in scope: every subsequent `Database(db\_path)` construction throws ? the app can never start against that DB, with no recovery path in code.
- Triggers are realistic, not purely hypothetical: SIGKILL/power-loss during the one-time startup upgrade, or ? notably ? an `OperationalError` raised by a ****later statement in the same block****. For v4, the block contains two ALTERs followed by index creation; if the second ALTER or an index creation hits `database is locked` (busy_timeout exhausted under the very concurrent-writer model these comments anticipate) or `disk I/O error`, the first ALTER is already auto-committed and the version never bumps. That permanently bricks the DB on next open. This is a code-internal trigger, not just external power loss.
- Single-statement window is narrow, but the "fail on a later statement in the same multi-statement block" window (v4 has 4 statements after the first ALTER) is materially wider and is exactly what busy_timeout/disk errors target.

****Arguments to REFUTE (intended / guarded / unreachable):****

- This is a ***one-time*** migration that runs only when crossing v3/v4, on a DB that pre-dates those versions. The corpus of such DBs is small and the window per open is narrow. But "small probability" ? "not a defect" ? the failure is permanent and unrecoverable when it does hit.
- It is single-tenant / local today (no concurrent writers yet), so the busy-lock trigger is mostly latent. However the code itself (busy_timeout, WAL, claim_due_job, "concurrent workers") is explicitly built for the async-job concurrent-writer world, and the v3 comment literally describes multiple workers ? so the lock-during-migration path is a designed-for reality, not a stretch.
- No test guards the interrupted case; the existing tests only cover happy-path and full-re-run idempotency, so this is NOT already-tested.

The mechanism is proven, the intent (crash-safe ladder) is established by the sibling blocks, it is not guarded or tested, and a code-internal trigger (a later-statement `OperationalError`, including lock exhaustion under the concurrency the module is built for) reaches it. Under the REFUTE lens I cannot honestly refute it ? it is a real, if low-frequency, defect with severe blast radius. I judge it real. Severity is appropriately low-to-medium given the narrow probability window but unrecoverable outcome.

19. user (2026-06-18T01:41:59.781Z)

Structured output provided successfully

20. assistant (2026-06-18T01:42:12.522Z)

I've completed my adversarial analysis of this candidate.

****Verdict: REAL (confirmed under the REFUTE lens, confidence ~0.6).****

I reproduced both halves of the mechanism on Python 3.13.13 / SQLite 3.50.4:

1. A bare ``ALTER TABLE ... ADD COLUMN`` auto-commits independently of the ``with conn:`` rollback scope in ``_connect`` ? when I raised an exception after the first ALTER inside the block, the column persisted while ``PRAGMA user_version`` stayed unbumped.
2. The next ``Database()`` open re-runs the same block and raises ``sqlite3.OperationalError: duplicate column name``, which propagates out of ``_init_db`` ? ``__init__``, permanently bricking the DB (constructed at startup in ``backend/main.py:782``).

Key file+lines: ``backend/infrastructure/database.py`` ? v3 block lines 156-159, v4 block lines 168-178.

What survived the refutation attempt:

- The v1/v2/v5 blocks are all ``CREATE ... IF NOT EXISTS`` (crash-safe), proving a re-runnable ladder is the design contract ? so the v3/v4 bare ALTERs are a genuine inconsistency, not intended behavior.
- Not guarded and not tested: ``tests/test_database.py`` covers only happy-path migration and full-re-run idempotency, never an interrupted mid-block migration.
- A code-internal trigger widens the window beyond mere power loss: an ``OperationalError`` (lock-timeout exhaustion / disk I/O error) on any **later** statement in the same v4 block (two ALTERs + two index creations follow the first ALTER) leaves the first ALTER committed and the version unbumped. The module is explicitly built for the concurrent-writer async-job world.

Severity is low-to-medium: narrow per-open probability window (one-time legacy v3/v4 upgrade, mostly single-tenant today) but an unrecoverable, no-code-recovery outcome. The proposed fix (per-column ``PRAGMA table_info`` existence guard before each ALTER, or atomic block scoping, or catch/continue on duplicate-column) is correct.